Detailed Review of PixelRNN Architecture

Contents

1 Introduction to PixelRNN

PixelRNN is a two-dimensional autoregressive model designed for image generation. The model predicts each pixel value in an image by conditioning on previously generated pixels, following a raster-scan order, typically from left-to-right and top-to-bottom. Each pixel intensity is modeled as a probability distribution conditioned on previous pixels, allowing PixelRNN to capture complex dependencies across pixels.

The model comprises layers of LSTM units adapted to two-dimensional spatial data, using specialized layers like Row LSTM and Diagonal BiLSTM to capture dependencies effectively across rows and diagonals. Additionally, residual connections help stabilize training in deeper networks.

2 Autoregressive Modeling in PixelRNN

For a given image I with pixel values $\mathbf{x}_{i,j}$ at location (i, j), PixelRNN models the joint probability distribution as follows:

$$p(I) = \prod_{i=1}^{H} \prod_{j=1}^{W} p(\mathbf{x}_{i,j} | \mathbf{x}_{<i,j}), \tag{1}$$

where H and W denote the height and width of the image, and $\mathbf{x}_{<i,j}$ represents all pixels that precede (i, j) in raster scan order. This autoregressive formulation enables PixelRNN to predict each pixel based on those generated prior to it.

3 Row LSTM Layer

The **Row LSTM** layer processes each row of the image sequentially from left to right, capturing horizontal dependencies. For each row r and pixel j in that row, the Row LSTM updates its hidden state $h_{r,j}$ based on the previous hidden state $h_{r,j-1}$ and the current pixel input $x_{r,j}$:

$$h_{r,j} = \text{LSTM}(x_{r,j}, h_{r,j-1}), \tag{2}$$

where the LSTM cell processes each pixel along the row, capturing dependencies among pixels within a row. The hidden state $h_{r,j}$ carries information about all pixels to the left of j in row r, which provides horizontal context.

3.1 Intuition

The Row LSTM acts like a memory along each row, where each pixel can "see" pixels to its left. This is beneficial for learning features such as object boundaries and horizontal textures.

4 Diagonal BiLSTM Layer

The **Diagonal BiLSTM** layer captures dependencies across the image diagonally, allowing the model to incorporate horizontal and vertical dependencies.

For a pixel at (i, j), the Diagonal BiLSTM updates its hidden state $h_{i,j}$ by considering pixels from both the previous row and previous column:

$$h_{i,j} = \text{BiLSTM}(x_{i,j}, h_{i-1,j-1}, h_{i-1,j+1}), \tag{3}$$

which allows bidirectional processing across rows and columns, capturing complex spatial dependencies.

4.1 Intuition

The Diagonal BiLSTM captures complex dependencies across the image by integrating information from multiple directions, beneficial for modeling textures, edges, and other spatial relationships.

5 Residual Connections

PixelRNN uses **residual connections** to stabilize training and improve convergence in deep networks. For each LSTM layer output h_l, the residual connection adds the layer's input to its output:

$$h_{l+1} = h_l + \text{LSTM}(h_l). \tag{4}$$

This technique preserves information across layers by passing features directly, enhancing training efficiency for networks with a deep architecture.

6 Output Layer and Multinomial Distribution

Each pixel value is treated as a discrete category using a multinomial distribution. For color images, the RGB channels are predicted jointly, with each channel conditioned on the other. The softmax layer provides a probability distribution for each channel:

$$p(\mathbf{x}_{i,j}) = \text{Softmax}(W h_{i,j} + b), \tag{5}$$

where W and b are learnable parameters that map the hidden state $h_{i,j}$ to output probabilities.

6.1 Intuition

Using a multinomial distribution allows PixelRNN to model pixel intensities naturally as discrete categories, leading to better learning outcomes in color images.

7 PixelCNN as an Alternative

PixelCNN simplifies PixelRNN by replacing LSTM layers with masked convolutions, which ensure that each pixel prediction is based only on preceding pixels.

7.1 Masked Convolutions

In PixelCNN, masks are applied to convolutional filters to prevent each pixel from seeing "future" pixels. The convolutional operation can be written as:

$$h_{i,j} = \sum_{k,l} W_{k,l} \cdot x_{i-k,j-l} \quad \text{subject to } k \leq i \text{ and } l \leq j, \tag{6}$$

where W is the convolutional kernel and the mask restricts it to positions preceding (i, j).

7.2 Fully Convolutional Structure

PixelCNN maintains spatial resolution throughout, using no pooling layers. Masked convolutions enable the network to capture spatial dependencies while being computationally efficient.

7.3 Intuition

The PixelCNN architecture allows efficient training and evaluation by leveraging the convolutional layer's parallelism, albeit at a slight trade-off in the expressiveness compared to PixelRNN.

8 Conclusion

PixelRNN and PixelCNN represent two powerful autoregressive architectures for image generation, each with unique approaches to capturing spatial dependencies. PixelRNN's Row LSTM and Diagonal BiLSTM layers enable rich, two-dimensional context across pixels, while residual connections stabilize deeper networks. PixelCNN provides a simpler alternative using masked convolutions, which enhances training efficiency but retains the ability to model dependencies across the image.

PixelRNN is well-suited for tasks requiring intricate spatial dependency modeling, while PixelCNN provides computational advantages with competitive performance in scenarios favoring parallel processing.

9 Introduction to LSTM

The Long Short-Term Memory (LSTM) network is a type of recurrent neural network (RNN) designed to address the issue of *vanishing gradients* that can occur in standard RNNs. Proposed by Hochreiter and Schmidhuber in 1997, the LSTM incorporates memory cells and gating mechanisms to enable the network to capture long-term dependencies and retain relevant information over extended time sequences.

10 Structure of an LSTM Cell

An LSTM cell is structured with three primary gates:

- **Forget Gate** f_t: Controls what portion of the previous cell state C_{t-1} should be retained.
- **Input Gate** i_t: Determines the extent to which new information $\tilde{C}_t$ is added to the cell state.
- **Output Gate** o_t: Regulates the amount of information from the cell state C_t that flows into the hidden state h_t.

The LSTM cell state C_t acts as a memory that carries information over time steps, while the hidden state h_t represents the cell's output at each step.

11 Mathematical Formulation

Let x_t denote the input vector at time step t, h_{t-1} the hidden state from the previous time step, and C_{t-1} the cell state from the previous time step. The LSTM equations are as follows:

11.1 Forget Gate

The forget gate f_t decides how much of the previous cell state C_{t-1} to retain:

$$f_t = \sigma\left(W_f \cdot \begin{bmatrix} h_{t-1} \\ x_t \end{bmatrix} + b_f\right), \tag{7}$$

where:

- W_f is the weight matrix for the forget gate,
- b_f is the bias term for the forget gate,
- σ is the sigmoid activation function, which outputs values between 0 and 1.

The forget gate's output f_t is a vector of values between 0 and 1, where each element indicates how much of each component of C_{t-1} to retain.

11.2 Input Gate

The input gate i_t controls how much new information $\tilde{C}_t$ is added to the cell state:

$$i_t = \sigma\left(W_i \cdot \begin{bmatrix} h_{t-1} \\ x_t \end{bmatrix} + b_i\right), \tag{8}$$

where:

- W_i is the weight matrix for the input gate,
- b_i is the bias term for the input gate.

The input gate's output i_t determines which parts of the candidate cell state $\tilde{C}_t$ to add to the current cell state C_t.

11.3 Candidate Cell State

The candidate cell state $\tilde{C}_t$ is a vector of new information that could be added to the cell state:

$$\tilde{C}_t = \tanh\left(W_C \cdot \begin{bmatrix} h_{t-1} \\ x_t \end{bmatrix} + b_C\right), \tag{9}$$

where:

- W_C is the weight matrix for the candidate cell state,
- b_C is the bias term for the candidate cell state,
- tanh is the hyperbolic tangent function, which outputs values between -1 and 1.

The candidate cell state $\tilde{C}_t$ represents potential new information that the LSTM can add to the current cell state.

11.4 Cell State Update

The current cell state C_t is updated by combining the previous cell state C_{t-1} (modulated by the forget gate) and the candidate cell state $\tilde{C}_t$ (modulated by the input gate):

$$C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t, \tag{10}$$

where $\odot$ represents element-wise multiplication. This equation allows the LSTM to selectively retain past information and add new information.

11.5 Output Gate and Hidden State

The output gate o_t controls how much of the cell state C_t is passed to the hidden state h_t, which is also the output of the LSTM cell:

$$o_t = \sigma\left(W_o \cdot \begin{bmatrix} h_{t-1} \\ x_t \end{bmatrix} + b_o\right), \tag{11}$$

where:

- W_o is the weight matrix for the output gate,
- b_o is the bias term for the output gate.

The hidden state h_t is then computed as:

$$h_t = o_t \odot \tanh(C_t). \tag{12}$$

The hidden state h_t provides a representation of the current time step's output, which can be passed to the next time step in a sequence or to other network layers.

12 Summary of LSTM Computation Steps

To summarize the steps of an LSTM cell:

1. Compute the forget gate f_t.
2. Compute the input gate i_t.
3. Compute the candidate cell state $\tilde{C}_t$.
4. Update the cell state $C_t = f_t \odot C_{t-1} + i_t \odot \tilde{C}_t$.
5. Compute the output gate o_t.
6. Compute the hidden state $h_t = o_t \odot \tanh(C_t)$.

These steps are repeated at each time step t in the sequence, allowing the LSTM to capture long-range dependencies by retaining relevant information in the cell state C_t.

13 Intuition and Advantages of LSTM

The main advantage of the LSTM is its ability to control the flow of information through time using the gating mechanisms, which helps it mitigate the vanishing gradient problem. The cell state C_t acts as a memory that can retain information across long sequences, while the forget, input, and output gates modulate which information is kept, updated, or revealed. This flexibility allows LSTMs to learn dependencies over long time periods, making them suitable for tasks such as language modeling, sequence prediction, and time-series forecasting.

14 Conclusion

The LSTM layer represents an effective solution to the challenge of learning long-term dependencies in sequential data. By maintaining a cell state and controlling information flow with gates, the LSTM can avoid the vanishing gradient problem and retain important information over extended sequences. This detailed mathematical framework provides the basis for understanding how LSTMs capture complex temporal patterns in data.